

SEAPLACE · FRONTEND

Migración del frontend a React + Vite

Qué se hizo, cómo se hizo y por qué se hizo así. Una guía para entender el proyecto desde cero y poder seguir trabajando solo.

Proyecto: `frontend-react/` dentro del repo SeaPlace

Backend: Spring Boot (sin cambios)

Fecha: 30 de septiembre de 2026

Contenido

1. Resumen: qué cambió
2. Los conceptos que necesitás
3. Cómo se armó el proyecto, paso a paso
4. Estructura de carpetas
5. De HTML a React: cómo se convirtió cada página
6. Conexión con Spring Boot
7. Decisiones tomadas y por qué
8. Qué mejoró respecto del HTML original
9. Cómo correr el proyecto
10. Cómo seguir: tareas comunes
11. Pendientes y próximos pasos
12. Problemas comunes y glosario

1. Resumen: qué cambió

Antes, el frontend eran **6 archivos HTML sueltos** en `frontend/`, cada uno con su propia copia del header, del footer y de la configuración de Tailwind, y un `cart.js` que manipulaba el DOM a mano. No se comunicaban con el backend.

Ahora hay una **aplicación React** en `frontend-react/`, creada con Vite, que:

- Se ve igual que el diseño original (mismos colores, tipografías, textos e imágenes).
- Tiene el header, el footer y cada tipo de tarjeta escritos **una sola vez** como componentes reutilizables.
- Navega entre páginas sin recargar el navegador (React Router).
- **Se conecta al backend de Spring:** el login y el registro son reales (JWT), y el catálogo pide los animales a `GET /animales`.
- Mantiene el carrito en `localStorage` como antes, pero manejado por React.

Antes (HTML)	Ahora (React)	URL
<code>pacifico/inicio.html</code>	<code>src/pages/Inicio.jsx</code>	<code>/</code>
<code>pacifico/catalogo.html</code>	<code>src/pages/Catalogo.jsx</code>	<code>/catalogo</code>
<code>pacifico/detalle-nori.html</code>	<code>src/pages/DetalleAnimal.jsx</code> (sirve para cualquier animal)	<code>/animales/:id</code>
<code>pacifico/carrito.html</code>	<code>src/pages/Carrito.jsx</code>	<code>/carrito</code>
<code>comun/login.html</code>	<code>src/pages/Login.jsx</code>	<code>/login</code>
<code>comun/dashboard.html</code>	<code>src/pages/Dashboard.jsx</code> (requiere login)	<code>/panel</code>
<code>comun/cart.js</code>	<code>src/context/CartContext.jsx</code>	—

La carpeta vieja no se borró

`frontend/` quedó intacta como referencia. Cuando estés conforme con la versión React, podés borrarla.

2. Los conceptos que necesitás

Node y npm

Node permite correr JavaScript fuera del navegador. **npm** es su gestor de paquetes (el equivalente a Maven en Java): lee `package.json`, que cumple el rol del `pom.xml`, y descarga las dependencias en `node_modules/`. Esa carpeta nunca se sube a git.

Vite

Es la herramienta que **corre** tu proyecto mientras programás y lo **empaqueta** para producción:

- `npm run dev` levanta `http://localhost:5173` y actualiza el navegador apenas guardás un archivo (sin recargar ni perder lo que tenías escrito).
- `npm run build` genera la carpeta `dist/` con HTML, CSS y JS optimizados, listos para subir a un servidor.

Los navegadores no entienden JSX. Vite lo traduce a JavaScript común al vuelo.

React y los componentes

React arma la interfaz con **componentes**: funciones de JavaScript que devuelven lo que se tiene que ver en pantalla. Un componente se usa como si fuera una etiqueta HTML nueva:

```
// Definición
function Icon({ name }) {
  return <span className="material-symbols-outlined">{name}</span>
}

// Uso (en cualquier otro componente)
<Icon name="pets" />
```

JSX

Es la sintaxis "HTML dentro de JavaScript" que ves en los `.jsx`. Se parece mucho al HTML, con dos diferencias clave:

- Entre `{ }` podés poner cualquier expresión de JavaScript: `{animal.nombre}`, `{precio * 2}`.
- Algunos atributos cambian de nombre. La lista completa está en la sección 5.

Props

Son los "parámetros" de un componente. En `<TarjetaCatalogo animal={luna} />`, el componente recibe `animal` y dibuja la tarjeta de Luna. Así, **una sola** definición de tarjeta sirve para todos los animales.

Estado (`useState`)

Es un dato que, cuando cambia, hace que React **vuelva a dibujar** el componente solo. Es la idea central de React:

```
const [busqueda, setBusqueda] = useState('') // valor inicial: ''  
  
<input value={busqueda} onChange={(e) => setBusqueda(e.target.value)} />
```

Antes, para filtrar el catálogo, había que recorrer las tarjetas con `querySelectorAll` y ocultarlas con `classList`. Ahora solo se cambia el estado, y la lista visible **se calcula** a partir de él.

Efectos (`useEffect`)

Sirven para hacer algo **después** de dibujar: pedir datos al backend, programar un temporizador, etc. Con `[]` como segundo argumento, el efecto se ejecuta una sola vez, cuando la página aparece. En el catálogo es donde se hace `fetch` de los animales.

Context

Permite compartir datos con **todos** los componentes sin pasarlos de padre a hijo. Se usan dos:

- `AuthContext`: quién está logueado, `login()`, `register()` y `logout()`.
- `CartContext`: el animal que está en el carrito. El número del header y la página del carrito lo leen de acá.

React Router

Hace que cada URL (`/catalogo`, `/carrito`...) muestre un componente distinto **sin recargar la página**. Este tipo de aplicación se llama SPA (*Single Page Application*). Los links internos se escriben `<Link to="/catalogo">` en lugar de `<a href="catalogo.html">`.

Tailwind CSS

Ya lo usabas: son las clases como `bg-primary` o `px-space-md`. Antes se cargaba desde un CDN, con la configuración copiada en cada HTML. Ahora está instalado como plugin de Vite y la configuración vive **una sola vez** en `src/index.css`.

3. Cómo se armó el proyecto, paso a paso

Estos son los comandos que se corrieron, en orden. Sirven para repetir el proceso en otro proyecto.

Paso 1: crear el proyecto con la plantilla de React

```
cd SeaPlace
npm create vite@latest frontend-react -- --template react
cd frontend-react
npm install
```

Esto genera `package.json`, `vite.config.js`, `index.html` y `src/` con un ejemplo. Los archivos de ejemplo (`App.css`, logos, `src/assets/`) se borraron.

Paso 2: instalar las librerías

```
npm install react-router-dom # navegación entre páginas
npm install -D tailwindcss @tailwindcss/vite # estilos (-D = solo para desarrollo/compilación)
```

Paso 3: configurar Vite

En `vite.config.js` se agregó el plugin de Tailwind y un **proxy** hacia Spring. El proxy se explica en la sección 6.

```
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
})
```

Paso 4: pasar el tema de Tailwind

El objeto `tailwind.config` que estaba repetido en cada HTML se tradujo a variables CSS dentro de `@theme`, en `src/index.css`. Tailwind v4 se configura así, desde CSS. Cada nombre genera sus clases automáticamente:

En <code>@theme</code>	Genera clases como
<code>--color-primary: #71523b</code>	<code>bg-primary</code> , <code>text-primary</code> , <code>ring-primary</code>
<code>--spacing-space-md: 1rem</code>	<code>p-space-md</code> , <code>gap-space-md</code>
<code>--font-headline-lg: "Plus Jakarta Sans"</code>	<code>font-headline-lg</code>
<code>--text-headline-lg: 2.25rem (+ line-height, peso)</code>	<code>text-headline-lg</code>

Las fuentes de Google (Be Vietnam Pro, Plus Jakarta Sans y Material Symbols) se cargan una sola vez en `index.html`.

Paso 5: la base compartida

1. `main.jsx`: monta la app y la envuelve en `BrowserRouter`, `AuthProvider` y `CartProvider`.
2. `App.jsx`: define qué página corresponde a cada URL.
3. `components/`: `Header`, `Footer`, `Layout`, `Icon`, `BarraProgreso` y `RutaProtegida`.
4. `api/`: todas las llamadas al backend.
5. `data/`: las URLs de las imágenes y los animales de ejemplo, que antes estaban escritos adentro de cada tarjeta HTML.

Paso 6: convertir las 6 páginas

Una por una, con el método de la sección 5.

Paso 7: verificar

```
npm run build      # compila: si hay errores de sintaxis, falla acá
npx oxlint src     # linter: detecta malas prácticas (quedó en 0 avisos)
```

Después se probaron con Spring corriendo, a través del proxy: el registro, el login correcto, el login con contraseña incorrecta (devuelve 401 con el mensaje de Spring) y `GET /animales`.

4. Estructura de carpetas

```
frontend-react/
├── index.html           ← única página HTML: carga fuentes y monta React en <div id="root">
├── package.json         ← dependencias y scripts (dev, build, lint)
├── vite.config.js       ← plugins + proxy /api → Spring
├── public/favicon.svg
└── src/
    ├── main.jsx         ← punto de entrada: Providers + Router
    ├── App.jsx          ← mapa de URLs → páginas
    ├── index.css        ← Tailwind + tema (colores, espaciados, tipografías)
    ├── api/
    │   ├── client.js    ← apiFetch: agrega /api, token JWT y maneja errores
    │   ├── tokenStorage.js ← guarda/lee/decodifica el JWT
    │   ├── auth.js      ← POST /auth/authenticate y /auth/register
    │   └── animales.js  ← GET /animales, /animales/{id}, /animales/{id}/fotos
    ├── context/
    │   ├── AuthContext.jsx ← sesión del usuario
    │   └── CartContext.jsx ← carrito (reemplaza a cart.js)
    ├── components/
    │   ├── Layout.jsx   ← Header + página actual + Footer; scroll al cambiar de URL
    │   ├── Header.jsx   ← menú con link activo automático y badge del carrito
    │   ├── Footer.jsx
    │   ├── Icon.jsx     ← <Icon name="pets" /> (Material Symbols)
    │   ├── BarraProgreso.jsx
    │   └── RutaProtegida.jsx ← si no hay sesión, redirige a /login
    ├── data/
    │   ├── imagenes.js  ← URLs de las fotos del diseño
    │   └── animales.js  ← animales de ejemplo + desdeApi() que adapta el JSON de Spring
    └── pages/
        ├── Inicio.jsx  Catalogo.jsx  DetalleAnimal.jsx
        └── Carrito.jsx  Login.jsx    Dashboard.jsx
```

Regla para ubicar las cosas

pages = una pantalla completa con su URL. **components** = piezas que usan varias páginas. **api** = único lugar que habla con el backend. **context** = datos globales. **data** = datos fijos.

5. De HTML a React: cómo se convirtió cada página

Cambios mecánicos (siempre los mismos)

HTML	JSX	Por qué
<code>class="..."</code>	<code>className="..."</code>	<code>class</code> es palabra reservada en JS
<code>for="email"</code>	<code>htmlFor="email"</code>	<code>for</code> también es reservada
<code>, <input ...></code>	<code></code>	en JSX toda etiqueta se cierra
<code>style="width: 68%;"</code>	<code>style={{ width: '68%' }}</code>	el estilo es un objeto JS
<code>onclick="selectAnimal(...)"</code>	<code>onClick={() => apadrinar(animal)}</code>	se pasa una función, no un texto
<code>checked="" / required=""</code>	<code>checked={x} / required</code>	atributos booleanos
<code>stroke-width, stop-color</code> (SVG)	<code>strokeWidth, stopColor</code>	camelCase
<code>maxlength, tabindex</code>	<code>maxLength, tabIndex</code>	camelCase
<code><!-- comentario --></code>	<code>{/* comentario */}</code>	sintaxis de JSX
<code></code>	<code><Link to="/carrito"></code>	navega sin recargar
<code>data-alt="..."</code>	<code>alt="..."</code>	accesibilidad (era un pendiente)

Cambio de mentalidad: de "tocar el DOM" a "cambiar el estado"

Este es el cambio importante. Un ejemplo real, el acordeón de preguntas frecuentes:

ANTES (JS + DOM)

```
function toggleFaq(button) {
  const answer = button.nextElementSibling;
  const chevron =
    button.querySelector('.faq-chevron');
  answer.classList.toggle('hidden');
  chevron.style.transform =
    answer.classList.contains('hidden')
      ? '' : 'rotate(180deg)';
}
```

AHORA (REACT)

```
const [abierta, setAbierta] = useState(null)

<button onClick={() =>
  setAbierta(abierta === i ? null : i)}>
  ...
</button>

{abierta === i && <p>{item.r}</p>}
```

En React no se busca ni se modifica ningún elemento. Se guarda **qué pregunta está abierta** y el JSX describe cómo se ve la pantalla para ese estado. React se encarga de actualizar el DOM.

Cambio de mentalidad: de "copiar bloques" a "recorrer datos"

El catálogo tenía 6 `<article>` casi idénticos. Ahora los datos viven en un array y se dibujan con `.map()`:

```
{visibles.map((animal) => (  
  <TarjetaCatalogo key={animal.id} animal={animal} onApadrinar={() => apadrinar(animal)} />  
))}
```

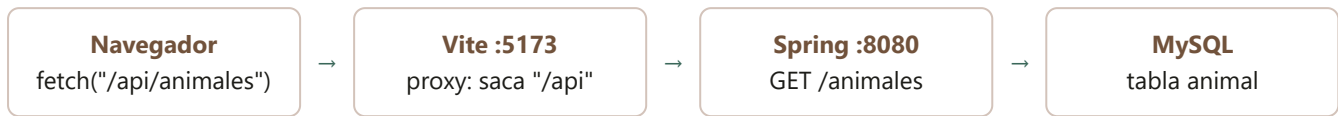
`key` es obligatorio en listas: le permite a React saber qué elemento es cuál cuando la lista cambia.

Página por página

Página	Qué estado maneja	Notas
Inicio	el toast de "¡seleccionado!"	Estadísticas, kit y testimonios pasaron a arrays con <code>.map()</code> .
Catálogo	animales, categoría, búsqueda, urgencia y orden	Pide los datos a la API; si no hay, usa los de ejemplo. Los números de cada categoría se calculan solos.
Detalle	foto activa, frecuencia, plan y regalo	Lee el <code>:id</code> de la URL con <code>useParams()</code> . Los precios de los 3 planes se calculan a partir de la cuota del animal.
Carrito	botiquín, certificado, método de pago, envío	El total se deriva del estado y ya no se recalcula a mano en un listener.
Login	modo (login/registro), formulario, error	Llama a Spring de verdad y muestra el mensaje de error del backend.
Panel	—	Protegido con <code>RutaProtegida</code> . Saluda con el nombre de usuario del token.

6. Conexión con Spring Boot

El recorrido de un pedido



¿Por qué un proxy y no CORS?

El navegador bloquea por seguridad que una página en `localhost:5173` llame a `localhost:8080`, porque son "orígenes" distintos. Hay dos soluciones:

1. **CORS**: configurar Spring para que acepte pedidos de otro origen. Implica tocar el backend (`SecurityConfig`).
2. **Proxy** (la elegida): el navegador le habla solo a Vite (mismo origen) y Vite le reenvía los pedidos a Spring. **No se tocó el backend**.

El prefijo `/api` sirve para que Vite distinga los pedidos al backend de las páginas del frontend: `/catalogo` es una página de React y `/api/animales` va a Spring.

Cómo funciona el login (JWT)

1. El formulario manda `{ nombreUsuario, contrasenia }` a `POST /api/auth/authenticate`.
2. Spring responde `{ "access_token": "eyJ..." }`.
3. El token se guarda en `localStorage` si se tildó "Mantener mi sesión abierta"; si no, en `sessionStorage`, que se borra al cerrar la pestaña.
4. El token tiene tres partes separadas por puntos. La del medio es JSON, y de ahí se lee `sub` (el nombre de usuario) y `exp` (el vencimiento). Si venció, se descarta.
5. `apiFetch` agrega `Authorization: Bearer <token>` a **todos** los pedidos siguientes.

Ojo: el login es con nombre de usuario, no con mail

El HTML original pedía "Correo electrónico", pero `AuthenticationRequest.java` espera `nombreUsuario`. El formulario se adaptó al backend. Para registrarse se piden los 5 campos de `RegisterRequest.java`: nombre, apellido, mail, nombre de usuario y contraseña.

Manejo de errores

El `GlobalExceptionHandler` de Spring siempre responde `{ "status": ..., "mensaje": "..." }`. `apiFetch` toma ese `mensaje` y lanza un error con ese texto, y la página lo muestra. Por eso, si el usuario ya existe, el login dice exactamente lo que dijo el backend.

Endpoints que usa el frontend hoy

Endpoint	Dónde se usa
POST /auth/authenticate	Login
POST /auth/register	Crear cuenta
GET /animales	Catálogo
GET /animales/{id} y /animales/{id}/fotos	Detalle de un animal que vino de la API

Datos reales vs. datos de ejemplo

Tu base todavía no tiene animales cargados: GET /animales devuelve una lista vacía. Por eso el catálogo usa los 10 animales de ejemplo y muestra un aviso. En cuanto cargues animales en la base, el catálogo pasa a mostrar los reales sin cambiar código. La función desdeApi() de data/animales.js traduce el AnimalResponse de Spring al formato que usan las tarjetas.

7. Decisiones tomadas y por qué

Decisión	Por qué
JavaScript y no TypeScript	Es más simple para arrancar desde cero. Si en la materia piden TypeScript, se puede migrar archivo por archivo.
Carpeta nueva <code>frontend-react/</code>	No se pierde nada del HTML original y se pueden comparar lado a lado.
Tailwind v4 instalado (no por CDN)	El CDN es solo para prototipos: es lento y no genera CSS optimizado. Instalado, el build incluye solo las clases que se usan (~64 KB).
Mismas clases y colores que el diseño	El objetivo era migrar la tecnología sin rediseñar. El tema se copió valor por valor.
React Router	Es el estándar para navegación en React. Además permite rutas protegidas (<code>/panel</code>) y parámetros (<code>/animales/:id</code>).
Context y no Redux	Hay solo dos datos globales (sesión y carrito). Redux sumaría mucha estructura para un problema chico.
Proxy de Vite y no CORS	No hubo que tocar el backend (ver sección 6).
Toda llamada al backend pasa por <code>api/</code>	Si cambia la URL, el token o el formato de error, se cambia en un solo lugar.
Datos de ejemplo como respaldo	La base está vacía y el sitio tiene que verse completo igual. También sirve si el backend está apagado.
Carrito en <code>localStorage</code>	El backend tiene <code>/carritos</code> , pero pide <code>idUsuario</code> , y el token solo trae el nombre de usuario. No hay forma de obtener el id sin tocar el backend (ver sección 11).
Misma clave de <code>localStorage</code> que <code>cart.js</code>	Si ya había algo en el carrito, no se pierde.
Un solo componente de detalle	Antes existía solo la página de Nori. Ahora <code>/animales/:id</code> sirve para cualquier animal. Nori conserva sus bloques exclusivos (ficha clínica, mapa y webcam).
Nombres en español	Coincide con el backend (<code>nombreAnimal</code> , <code>cuotaApadrinamiento</code>) y con el resto del proyecto.

8. Qué mejoró respecto del HTML original

- El select "**Ordenar**" del catálogo ahora funciona (era un pendiente del resumen anterior).
- Accesibilidad:** los `data-alt` pasaron a ser `alt` reales (el otro pendiente). También se agregaron `aria-pressed`, `aria-expanded` y etiquetas en los campos.

- Las **miniaturas** del detalle cambian la foto grande de verdad; antes solo movían el borde.
- El botón "**Historia**" del catálogo lleva al detalle del animal; antes no hacía nada.
- Los contadores de las categorías ("Focas 12", "Nutrias 6"...) se calculan solos; antes estaban escritos a mano y no coincidían con las tarjetas.
- El link del menú resaltado corresponde siempre a la página actual.
- El panel (/panel) exige estar logueado y saluda con tu nombre de usuario.
- Se reemplazaron dos imágenes del dashboard que tenían URLs inválidas y no cargaban.
- El newsletter del footer muestra una confirmación.
- Al pasar de una página a otra, la vista vuelve arriba, y los links a secciones (/#kit-protector) funcionan.

Lo que se sacó porque era decorativo y no hacía nada: los botones de paginación 1-2-3 del catálogo y los botones "Descargar tarjeta regalo" / "Ver muestra digital". Se reemplazaron por un link a un detalle, donde sí se puede marcar como regalo.

Un dato que se unificó: Nori figuraba a \$15 en el inicio y a \$24 en su detalle. Ahora tiene una sola cuota (\$24) en todo el sitio.

9. Cómo correr el proyecto

La primera vez (o después de clonar el repo)

```
cd frontend-react
npm install          # descarga node_modules (solo hace falta una vez, o cuando cambia
package.json)
```

Cada vez que vas a trabajar: dos terminales

```
# Terminal 1 – backend (desde la raíz del repo; necesita MySQL corriendo)
./mvnw spring-boot:run          # o ejecutar SeaPlaceApplication desde IntelliJ

# Terminal 2 – frontend
cd frontend-react
npm run dev
```

Abri **http://localhost:5173**. Si el backend no está levantado, el sitio anda igual con los datos de ejemplo, pero el login falla con "No se pudo conectar con el servidor".

Comandos disponibles

Comando	Qué hace
<code>npm run dev</code>	Servidor de desarrollo con recarga instantánea
<code>npm run build</code>	Genera <code>dist/</code> para producción
<code>npm run preview</code>	Sirve <code>dist/</code> para probar el build
<code>npm run lint</code>	Corre el linter (oxlint)

10. Cómo seguir: tareas comunes

Agregar una página nueva (ej. "Nuestra Misión")

1. Crear `src/pages/Mision.jsx`:

```
export default function Mision() {
  return <h1 className="font-headline-lg text-headline-lg text-primary">Nuestra Misión</h1>
}
```

2. Registrarla en `App.jsx`, dentro del `<Route element={...}>`:

```
<Route path="/mision" element={...} />
```

3. Linkearla desde `Header.jsx`: `<NavLink to="/mision" className={...}>Nuestra Misión</NavLink>`.

Llamar a un endpoint nuevo

Agregá una función en `src/api/` y usala desde la página. El token y los errores ya los maneja `apiFetch`:

```
// src/api/categorias.js
import { apiFetch } from './client'
export function getCategorias() {
  return apiFetch('/categorias')
}
```

Proteger una página

```
<Route path="/mis-compras" element={<RutaProtegida><MisCompras /></RutaProtegida>} />
```

Usar la sesión o el carrito en cualquier componente

```
const { usuario, logout } = useAuth()
const { item, setSponsorship, clearSponsorship } = useCart()
```

11. Pendientes y próximos pasos

Lo más importante: falta un endpoint en el backend

Para conectar el carrito y la compra de verdad (`POST /carritos/items` y `POST /compras`), el frontend necesita el `idUsuario` de quien está logueado. Hoy el token solo trae el nombre de usuario, y `GET /usuarios` está restringido a administradores. La solución más simple es agregar `GET /usuarios/me`, que devuelva el `UsuarioResponse` del usuario autenticado. En `Carrito.jsx`, la función `confirmar()` tiene un `TODO` marcando dónde irían esas llamadas.

1. **Cargar animales en la base** (por `data.sql` o con `POST /animales`) para que el catálogo muestre datos reales.
2. **Fotos:** los animales de la API usan una foto genérica hasta que se carguen en `/animales/{id}/fotos`.
3. **Panel del protector:** mostrar los apadrinamientos reales con `GET /compras?idUsuario=...`, que también necesita el id.
4. **Menú en celulares:** el diseño original ocultaba el menú en pantallas chicas y no tenía botón "hamburguesa". Sigue igual.
5. **Producción:** `npm run build` genera `dist/`. Se puede servir con cualquier hosting estático (Netlify, Vercel) apuntando a la URL real del backend, o copiarlo a `src/main/resources/static` para que lo sirva Spring.
6. **Limpieza:** cuando estés conforme, borra la carpeta `frontend/` vieja y el usuario de prueba `prueba_react` que se creó en tu base local para verificar el registro.

12. Problemas comunes

Síntoma	Causa y solución
"No se pudo conectar con el servidor" al loguearte	Spring no está corriendo, o MySQL está apagado. Levántalos y reintentá.
'vite' no se reconoce como comando	Falta <code>npm install</code> dentro de <code>frontend-react</code> .
Una clase de Tailwind no aplica	Revisá que el nombre exista en <code>@theme (index.css)</code> . Tailwind v4 cambió algunos nombres: por ejemplo, <code>bg-gradient-to-r</code> ahora es <code>bg-linear-to-r</code> .
Pantalla en blanco	Abrió la consola del navegador (F12). Casi siempre es un error de JavaScript con el archivo y la línea.
Recargo en <code>/catalogo</code> y da 404 en producción	En una SPA, el servidor tiene que devolver <code>index.html</code> para cualquier ruta. En desarrollo Vite ya lo hace; en producción hay que configurarlo en el hosting.
Siempre aparecen los "animales de ejemplo"	Es lo esperado mientras la tabla <code>animal</code> esté vacía o el backend esté apagado.

Glosario rápido

Componente	Función que devuelve JSX. Es una pieza reutilizable de la interfaz.
Props	Datos que un componente recibe de su padre (como parámetros).
Estado	Dato interno que, al cambiar, redibuja el componente (<code>useState</code>).
Hook	Función de React que empieza con <code>use</code> (<code>useState</code> , <code>useEffect</code> , <code>useAuth</code> ...). Solo se llama en el nivel superior de un componente.
SPA	Aplicación de una sola página: el navegador carga <code>index.html</code> una vez y React cambia el contenido.
Proxy	Intermediario: Vite recibe los pedidos a <code>/api</code> y se los pasa a Spring.
JWT	Token firmado por el backend que identifica al usuario en cada pedido.
Build	Versión optimizada del sitio lista para publicar (<code>dist/</code>).